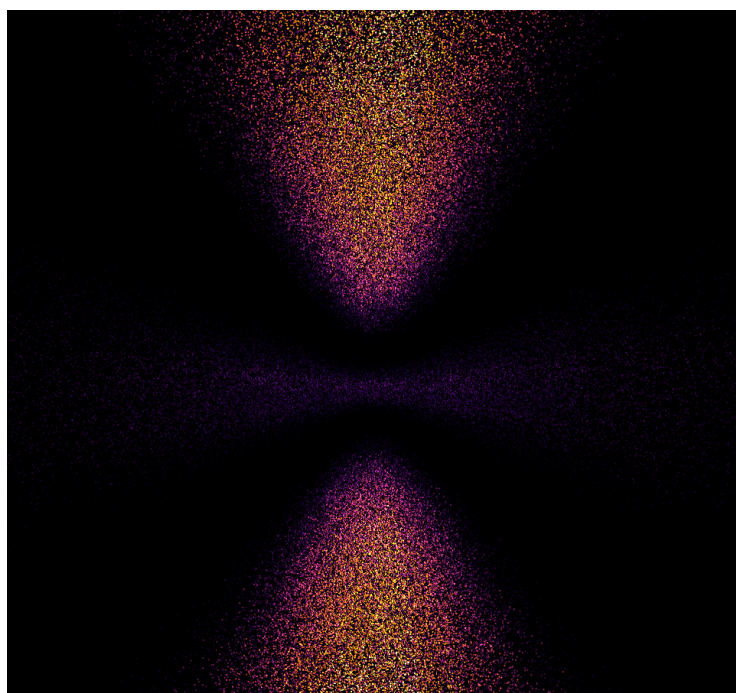


NEXT Sukkertoppen HTX
3.W HTX - Programmering B

Kvantemekanisk Simulering af Hydrogen Atomets Orbitaler

Et interaktiv partikelvisualisering af bølgefunktionen $\psi(n, l, m)$



Forfatter: Khizar Irshad Chaudhry
Klasse: 3.W HTX
Fag: Programmering B — Eksamensprojekt
Skole: NEXT Sukkertoppen HTX
Afleveret: April 2026

Resumé

Dette eksamensprojekt dokumenterer udviklingen af et interaktivt visualiseringsprogram til hydrogenatomets orbitaler, skrevet i Python og renderet gennem OpenGL (PyOpenGL + GLFW). Programmet sampler elektronens sandsynlighedsfordeling via en CDF-baseret algoritme ud fra bølgefunktionen $\psi(n, l, m)$ og tegner resultatet som en partikelsky, hvor hver partikel får tildelt en farve ud fra den lokale tæthed. Brugeren kan dynamisk ændre de tre kvantetal n , l og m samt antallet af partikler via en selvbygget HUD.

Da opgaven er stillet i faget **Programmering B**, holder synopsen den kvantemekaniske teori på et minimum og fokuserer i stedet på de programmeringstekniske valg. Til en dybere forståelse, er der valgt tre dybdeområder. (1) den objektorienterede **Electron**-klasse, (2) GPU-pipelinen med VAO/VBO og GLSL shaders, og (3) CDF samplingsalgoritmen. Den største læringskurve lå i selv at skulle allokere buffere på GPU'en og skrive shaders fra bunden.

Indhold

1	Indledning	3
2	Problemformulering	3
3	Kravsifikation	4
4	Kort teknisk baggrund	4
4.1	Bølgefunktionen	4
4.2	CDF sampling: matematik til punkter	4
5	Programmets arkitektur	5
5.1	Moduloversigt	5
5.2	Klassediagram	5
5.3	Dataflow pr. frame	5
6	Fokusområder	6
6.1	Dybde 1: CDF sampling	6
6.1.1	Pseudokode	6
6.1.2	Implementering	7
6.2	Dybde 2: OOP design af Electron klassen	8
6.2.1	Klassedefinition: Electron	8
6.2.2	Hvorfor en klasse?	9
6.2.3	Kode	9
6.3	Dybde 3: GPU-pipelinen med VAO, VBO og shaders	10
6.3.1	Begrebsafklaring	10
6.3.2	Opsætning	11
6.3.3	Upload af partikler m. dynamisk buffer	11
6.3.4	Shaders (GLSL)	11
7	Udviklingsprocessen	12
7.1	Milepæle	12
7.2	Git	13
7.3	Udfordringer undervejs	13
8	Test	14
9	Diskussion	14
10	Konklusion	15
11	Perspektivering	15
A	Bilag - Kodebasen	17

1 Indledning

I gymnasiets fysikundervisning introduceres atomet typisk gennem Bohr modellen, hvor elektroner kredser om atomkernen i faste, cirkulære baner. I mit studieområdeprojekt (SOP), "*Kvantemekanisk Sammenfiltrering*" [1] undersøgte jeg den kvantemekaniske virkelighed bag partiklerne, hvor bl.a. en elektron i stedet beskrives ved en *bølgefunktion* $\psi(r, \theta, \varphi)$. Elektronens position er ikke deterministisk, men en *sandsynlighedssky*, hvis tæthed er givet ved $|\psi|^2$.

Problemet er, at denne sandsynlighedssky er svær at forestille sig ud fra ligninger på et whiteboard, hvilket jeg især oplevede under arbejdet med mit SOP projekt. Jeg ville derfor gerne lave et program, hvor man kan opleve og *dreje rundt på* elektronskyen, hvor man selv kan ændre kvantetallene n , l , m og se, hvordan orbitalens form og størrelse ændrer sig i realtid. Inspirationen og grundlæggende teori til projektet kom fra YouTube-kanalen kavan010 [2], men jeg har implementeret hele programmet fra bunden i Python. Den grafiske del af programmet, herunder shader koden og OpenGL kaldene er inspireret af officiel OpenGL dokumentation [3], StackOverflow tråde samt generel internet hjælp til fejlfinding i GLSL sproget.

Dette er et **Programmering B**-projekt, og jeg har derfor valgt at placere fysikken i baggrunden og lade *programmeringen* være i centrum. Det har også været klart fra starten, at jeg ville bevæge mig væk fra det trygge p5.js-lag jeg brugte i mit Programmeringsprojekt (SVC) sidste år [8] og i stedet arbejde direkte mod GPU'en gennem OpenGL. Det har været den største teknologiske læringskurve, og en stor del af rapporten handler om, hvad det har betydet for designet.

2 Problemformulering

Hvordan kan man i Python designe og implementere et objektorienteret, interaktivt program, der visualiserer en hydrogenatom-elektrons sandsynlighedsfordeling som en partikelsky, og som renderer hundredtusindvis af partikler i realtid gennem OpenGL?

Den opstillede problemformulering ovenfor er bevidst programmeringsorienteret. Gennem synopsen her, er jeg interesseret i *hvordan* partiklerne kommer fra et matematisk udtryk og ind på GPU'ens buffer, og hvordan et OOP design kan holde kvantetalsreglerne på ét sted. I stedet for at fokusere på det bagvedliggende matematik og fysik¹.

Gennem rapporten vil følgende underspørgsmål befinde sig besvaret:

1. Hvordan oversættes en kontinuert sandsynlighedsfordeling $|\psi|^2$ til diskrete punkter til partikelsens mulige position?
2. Hvordan struktureres koden, så n , l og m altid overholder kvantemekanikkens regler ($l \in [0, n - 1]$, $m \in [-l, l]$), uanset hvad brugeren klikker på?
3. Hvordan bygges en OpenGL pipeline med VAO, VBO og GLSL shaders fra bunden i Python?
4. Hvordan undgår man, at programmet fryser, når brugeren trækker i slideren, der potentielt genererer 1.000.000 nye partikler?

¹På tords alt har jeg Matematik A og Fysik A, er jeg nødt til at begrænse projektets omfang til blot Programmering B.

3 Kravspecifikation

På baggrund af feedbacken fra min tidligere synopsis [?] (hvor jeg brugte lærerens krav i stedet for mine egne), har jeg her formuleret kravene selv. Det er naturligt at danne nogle krav til et projekt. Ellers er det en vej uden destination man følger. Blot derfor opstilles et kravspecifikation til projektet. Kravspecifikationen opdeles i *need-to-have* og *nice-to-have* med ID'er, så jeg kan referere tilbage i testafsnittet.

ID	Type	Krav
K1	Need	En partikelsky skal vise $ \psi(n, l, m) ^2$ ved at rendere et stort antal sampled positioner som punkter.
K2	Need	En Electron -klasse skal indkapsle n, l, m , samt håndhæve de fysiske regler automatisk.
K3	Need	Brugeren skal dynamisk kunne ændre n, l, m via tastaturet i realtid.
K4	Nice	Der skal være en HUD-slider til at ændre antal partikler.
K5	Nice	Aktuelle værdier af n, l, m og antal partikler skal vises på skærmen.
K6	Nice	Partiklerne skal farvelægges efter <i>lokal tæthed</i> (høj tæthed = lys/varm farve).
K7	Nice	Alpha værdien pr. partikel skal skaleres med tætheden, så sjældne positioner er svagt synlige.
K8	Nice	Kameraet skal kunne roteres og zoomes med musen.

Tabel 1: Egne krav til programmet med ID'er. Alle K1–K8 er i dag opfyldt.

4 Kort teknisk baggrund

Dette afsnit holder jeg *bevidst kort*, da fokus ligger på programmeringen. Den fulde kvantemekaniske teori samt udledning findes i mit SOP [1].

4.1 Bølgefunktionen

For et hydrogenatom kan ψ opsplittes i en radial og en angulær del:

$$\psi(r, \theta, \varphi) = R_{nl}(r) \cdot Y_l^m(\theta, \varphi).$$

R_{nl} er bygget på Laguerre-polynomier, Y_l^m på sfæriske harmonier. Sandsynligheden for at finde elektronen i et lille volumen er $|\psi|^2 dV$. Det er *denne sandsynlighedsfordeling* vi i programmet oversætter til konkrete punkter.

4.2 CDF sampling: matematik til punkter

Problemet er, at man ikke bare kan "plukke"punkter direkte fra en formel. I stedet bruges **inverse transform sampling** (CDF sampling) [5]. Følgende steps er påkrævet for at udvøre CDF samplingen:

1. Evaluer sandsynlighedstætheden $P(x)$ i mange jævnt fordelte punkter.
2. Normaliser så $\sum P = 1$.
3. Beregn kumulativ sum (CDF), ligesom en trappe fra 0 til 1.
4. Træk et uniformt tal $u \in [0, 1]$.
5. Find det x , hvor u lander på trappen. Det er én sample.

Bemærk at CDF er en *algoritme*, ikke fysik, og den er kernen i programmet. Jeg uddyber implementationen i afsnit 6.1.

5 Programmets arkitektur

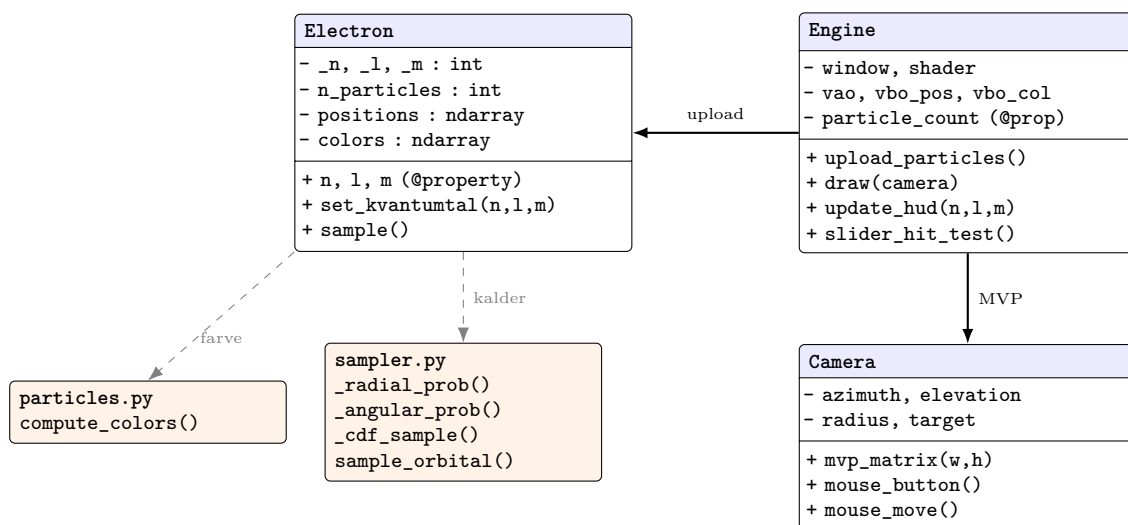
5.1 Moduloversigt

Programmet er opdelt i seks Python filer/moduler, hvor hvert modul har et klart ansvar:

Modul	Ansvar
main.py	Entry point. Opretter Engine , Camera , Electron . Binder GLFW-input til callbacks. Kører render-loopet.
engine.py	Alt OpenGL-arbejde: vindue, shaders, VAO, VBO, HUD overlay, slider.
camera.py	Sfærisk orbit kamera. Producerer MVP matrixen hver frame.
electron.py	OOP-klassen Electron ejer n , l , m og kalder sampling.
sampler.py	Den rene <i>algoritmiske</i> kerne til fysikken, hvor CDF sampling af r , θ , φ
particles.py	Tæthedsestimering via voxel grid og farvemapping (matplotlib colormap).

5.2 Klassediagram

Jeg har bevidst holdt klassestrukturen enkel med to klasser (**Engine** og **Electron**) samt **Camera**, plus en række "rene" moduler med funktioner (**sampler**, **particles**). På Figur 1 ses nogle klasse diagrammer², og deres relation til hinanden samt den samlede projekt.

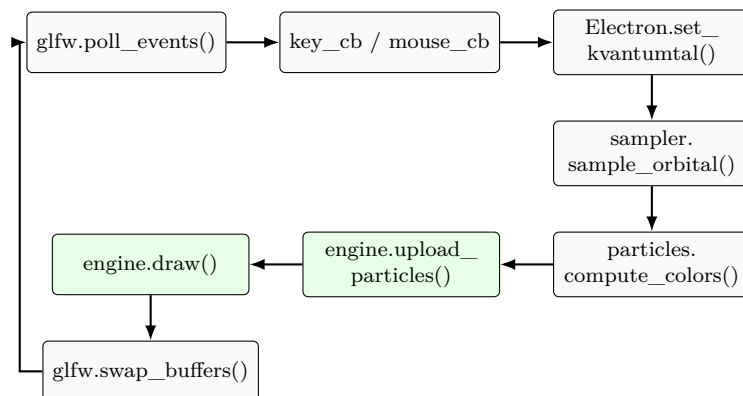


Figur 1: Klassediagram. Fyldte pile beskriver kompositions-/afhængighed mellem klasser gennem kald. Stiplede pile beskriver funktionskald til rene moduler.

5.3 Dataflow pr. frame

Programmet er et grafisk program, og derfor indeholder frames. Hver frame i render loopet det dataflow, der ses på diagrammet på Figur 2.

²Hvor i der også ses klassens attributes og hhv. variabel types, samt klassens metoder.



Figur 2: Dataflow pr. frame. Grønne bokse kører på GPU'en, resten på CPU'en.

6 Fokusområder

I min tidligere synopsis blev jeg kritiseret for at "favne bredt, men ikke dybt-?". Derfor har jeg denne gang valgt **tre dybdeområder** og afviser bevidst at beskrive alle dele af koden i samme detalje. De tre områder er valgt, fordi de samlet set dækker projektets største programmeringsudfordringer. Dette inkludere 1. den algoritmiske kerne (CDF), 2. det objektorienterede design (**Electron**), og 3. GPU-programmeringen (VAO/VBO/shaders).

6.1 Dybde 1: CDF sampling

CDF samplingen er algoritmen, der er ansvarlig for at bestemme de reale positioner (x , y , z) for partiklerne, ude fra sandsynlighedsfordelingen over rummet. Problemet er dog at sandsynlighedsfordelingen er en kontinuert matematisk funktion³, og vi kan ikke sample uendelige mange punkter. Derfor diskretiseres funktionen til 3000 jævnt fordelte punkter, som algoritmen kan behandle. Der benyttes det man kalder *Cantor funktion* [7]. CDF løser derfor problemet i at plukke diskrete antal positioner, i takt med at følge sandsynlighedsfordelingen. Løsningen er at omdanne sandsynlighedsfordelingen til en trappefunktion fra 0 til 1, hvor brede trapper svarer til høj sandsynlighed og smalle trapper til lav sandsynlighed. Ved at kaste tilfældige tal mod denne trappefunktion lander flere tal i de brede trapper. Algoritmen anvendes to gange, én gang for den radiale koordinat r og én gang for den polære vinkel θ . Den azimutale vinkel ϕ kræver ikke CDF sampling, da den er uniformt fordelt⁴. [6]

6.1.1 Pseudokode

```

funktion cdf_sample(x_vals, prob, n_samples):
    prob    <- |prob|
    prob    <- prob / sum(prob)           # normaliser
    cdf     <- kumulativ sum af prob      # trappe 0..1
    u       <- n_samples tilfældige tal i [0,1]
    indices <- for hvert u: find hvor u lander paa trappen
    returner x_vals[indices]
  
```

Pseudokoden ovenfor beskriver de fem trin i CDF sampling algoritmen. Først tages absolutværdien af sandsynlighederne for at garantere ikke-negative værdier, da floating point beregninger i

³Det betyder at den har værdi ved uendelige mange punkter. Dvs. $P(r) > 0$ ved ALLE r .

⁴Alle retninger rundt om z -aksen er lige sandsynlige.

bølgefunktionen kan producere meget små negative tal nær knudepunkterne. Derefter normaliseres sandsynlighederne ved at dividere med deres sum, så de tilsammen udgør 1.0, ellers ville sandsynlighedsfordelingen være ugyldig. Den kumulative sum omdanner herefter denne fordeling til en trappefunktion fra 0 til 1, hvor bredden er decideret sandsynligheden. Til sidst tegnes `n_samples` tilfældige tal mellem 0 og 1, og for hvert tal findes det trin på trappen det lander på, det tilsvarende `x_vals` indeks er den samplede position. Resultatet er et array af positioner der er fordelt præcis som den oprindelige sandsynlighedsfordeling beskriver.

6.1.2 Implementering

Den Python implementation jeg endte med er kun 8 linjer lang, men udnytter `numpy`'s vektoriserede operationer, så alle n samples beregnes på én gang. Den langsomme del er derfor ikke CDF udregningen, men selve evalueringen af $|\psi|^{25}$:

```

1 def _cdf_sample(x_vals, prob, n_samples):
2     prob = np.abs(prob)
3     prob /= prob.sum()           # 1) normaliser
4     cdf = np.cumsum(prob)        # 2) CDF
5     cdf /= cdf[-1]              # trappefunktion 0..1
6
7     u = np.random.uniform(0.0, 1.0, n_samples) # 3) tilfældige tal
8     indices = np.searchsorted(cdf, u)          # 4) find hvor u lander
9     indices = np.clip(indices, 0, len(x_vals) - 1)
10    return x_vals[indices]
```

Listing 1: Kerne-CDF-sampling fra `sampler.py`. Tre linjer gør alt arbejdet.

Blot ved at aflæse implementationen ses i Listing 1, dannes der nogle indlysende spørgsmål. Såsom:

Hvorfor `np.searchsorted`? CDF'en er monotont voksende, hvilket betyder at "find hvor u lander på trappen" er et klassisk binærsøgnings problem med tidskompleksitet $O(\log n)$ pr. opslag. I stedet for at skrive en search algoritme fra bunden, udnytter jeg funktionen `np.searchsorted`, der gør det vektoriseret for alle n samples på én gang, og endda direkte i C-sproget. Det er over $10\times$ hurtigere end en Python-for-løkke i egne målinger.

Hvordan bruges det? I `sample_orbital` kaldes `_cdf_sample` to gange. En gang for radialkoordinaten r (ikke-uniform fordeling) og én gang for polærvinklen θ . Azimutalvinklen φ er uniformt fordelt (ingen φ -afhængighed i $|\psi|^2$ for hydrogen), og kan derfor samples direkte via `np.random.uniform` uden CDF. Dette er en lille optimering, og samt viser at man ikke skal bruge den resourcekrævnede algoritme, når det ikke er nødvendigt.

⁵ $|\psi|^2$ afhænger af $R_{nl}(r)$ og $Y_l^m(\theta, \varphi)$.

6.2 Dybde 2: OOP design af Electron klassen

6.2.1 Klassedefinition: Electron

Klassen `Electron` repræsenterer én elektron i et hydrogen-lignende atom. Den samler kvantetallene (n, l, m) , de samplede positioner og farverne ét sted, og udgør broen mellem fysikberegningerne i `sampler.py` og GPU-renderingen i `engine.py`.

Konstruktør

Parameter	Type	Beskrivelse
<code>n</code>	<code>int</code>	Hovedkvantetallet — energiniveau, $n \geq 1$
<code>l</code>	<code>int</code>	Azimutal kvantetallet — orbitalform, $0 \leq l \leq n - 1$
<code>m</code>	<code>int</code>	Magnetisk kvantetallet — orientering, $-l \leq m \leq l$
<code>n_particles</code>	<code>int</code>	Antal punkter der samples fra bølgefunktionen

Konstruktøren sætter kvantetallene, initialiserer `positions` og `colors` til `None`, og kalder straks `sample()` så objektet altid er klar til brug efter oprettelse.

Properties

Kvantetallene `n`, `l` og `m` er read-only properties. De kan læses frit udefra, men kan kun ændres gennem `set_kvantumtal()`, som håndhæver de fysiske regler automatisk. Dette er et objektorienteret designvalg, også kaldt encapsulation, der forhindrer at kvantetallene sættes til ugyldige værdier direkte.

Metoder

`set_kvantumtal(n, l, m)` Opdaterer kvantetallene og re-sampler elektronen. Håndhæver automatisk reglerne:

$$\begin{aligned} n &= \max(1, n) \\ l &= \text{clamp}(l, 0, n - 1) \\ m &= \text{clamp}(m, -l, l) \end{aligned}$$

Kaldes fra `main.py` hver gang brugeren trykker en tast der ændrer et kvantetal.

`sample()` Kalder `sample_orbital(n, l, m, n_particles)` fra `sampler.py` for at sample nye positioner fra bølgefunktionen $\psi(n, l, m)$, og `compute_colors(positions)` fra `particles.py` for at beregne farver baseret på lokal tæthed. Resultaterne gemmes i `self.positions` og `self.colors` som numpy arrays af form $(N \times 3)$ og $(N \times 4)$.

Attributter

Attribut	Type	Beskrivelse
<code>positions</code>	<code>np.ndarray</code> <code>None</code>	$(N \times 3)$ float32 — xyz positioner i picometer
<code>colors</code>	<code>np.ndarray</code> <code>None</code>	$(N \times 4)$ float32 — RGBA farver per partikel
<code>n_particles</code>	<code>int</code>	Antal samplede punkter

6.2.2 Hvorfor en klasse?

Jeg kunne have implementeret elektronen som et dictionary, tuple, eller multi-dimensionelt array. Valget af en klasse er motiveret af *tre* konkrete designproblemer:

1. **Invarianter** — kvantemekanikken kræver følgende egenskaber ved kvantetalene: $n \geq 1$, $l \in [0, n - 1]$ og $m \in [-l, l]$. Hvis brugeren trykker på en tast, der ville bryde en regel, skal programmet ikke crashe, men *klampe* værdien automatisk. Selvom dette kan gøres gennem *if-statements*, og andre metoder. Så er klassen det naturlige sted at håndhæve disse invarianter ét sted.
2. **Indkapsling** — n , l og m skal være *read-only* udefra, ellers ville man kunne skrive `electron.n = 10` og omgå regelhåndhævelsen. Jeg bruger `@property`-decoratoren til at gøre attributterne til skrivebeskyttede properties udadtil, mens de underliggende `_n`, `_l`, `_m` er private pr. Python konvention (underscore-prefix).
3. **Single Responsibility Principle (SRP)** — `Electron` *ejer* kvantetal og positioner, men delegerer selve samplingen til `sampler.py` og farvemappingen til `particles.py`. Klassen ved intet om ψ eller voxel-grids. Princippet om Single Responsibility overholdes, og resultatet er *clean-code*.

6.2.3 Kode

```

1 class Electron:
2     def __init__(self, n, l, m, n_particles):
3         self.n_particles = n_particles
4         self._n, self._l, self._m = n, l, m
5         self.positions = None
6         self.colors = None
7         self.sample()
8
9     @property
10    def n(self) -> int: return self._n          # read-only udadtil
11    @property
12    def l(self) -> int: return self._l
13    @property
14    def m(self) -> int: return self._m
15
16    def set_kvantumtal(self, n, l, m):
17        """Eneste lovlig vej til at ændre kvantetal."""
18        self._n = max(1, n)                    # n >= 1
19        self._l = min(max(0, l), self._n - 1)   # 0 <= l <= n-1
20        self._m = max(-self._l, min(self._l, m)) # -l <= m <= l
21        self.sample()

```

Listing 2: Kerne af `Electron`-klassen: `@property` og invariant-håndhævelse.

Konsekvens i `main.py`. Fordi al logikken sidder i `set_kvantumtal`, bliver `key_cb`-handleren i `main.py` meget kortere, den behøver ikke validere noget selv:

```

1 if key == glfw.KEY_W: n += 1
2 elif key == glfw.KEY_S: n -= 1
3 # ...

```

```

4 if changed:
5     electron.set_kvantumtal(n, l, m)    # klassen klarer reglerne
6     engine.upload_particles(electron.positions, electron.colors)

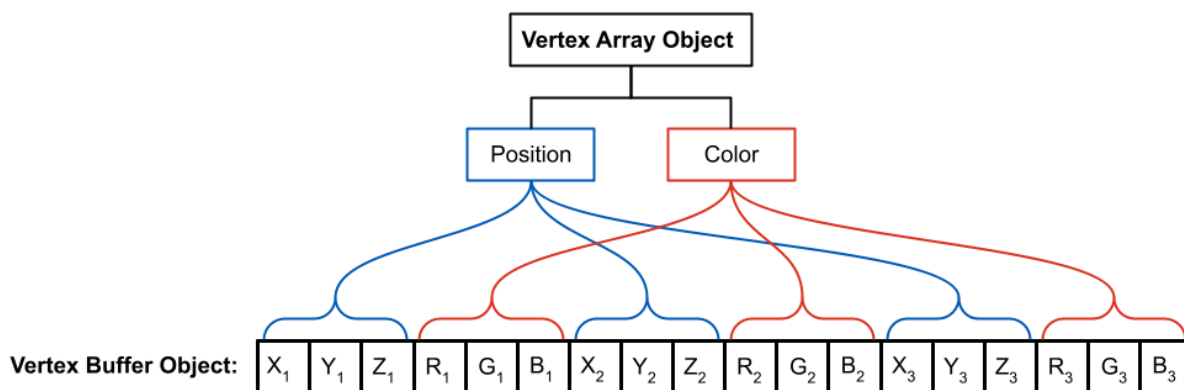
```

Hvis jeg senere vil tilføje flere regler (fx et maks- n for at undgå for tunge udregninger), skal jeg kun ændre ét sted.

6.3 Dybde 3: GPU-pipelinen med VAO, VBO og shaders

Dette har været den helt store læringskurve i projektet. Jeg havde aldrig brugt OpenGL før, og i modsætning til p5.js (som jeg brugte i min SVC (gravitations simulering) [8]) er OpenGL *lav-niveau*. Man allokerer selv buffere på GPU'en, skriver selv shader kode i GLSL og binder selv attributter til buffer offsets.

6.3.1 Begrebsafklaring



Figur 3: VBO er en lang buffer med bytes på GPU'en, der indeholder alt data. VAO er systemet der tager aflæser VBO og indeller informationen i rene datasæt og angiver attributes. På Figuren ses **Position** og **Color** som de to attributes.

VBO (Vertex Buffer Object) En rå buffer med bytes på GPU'en. Jeg bruger to: én til positioner ($N \times 3 \text{ float32}$) og én til farver ($N \times 4 \text{ float32}$).

VAO (Vertex Array Object) Et system der fortæller GPU'en, hvordan bytes i en VBO skal læses, ting som *hvor mange floats er én vertex, hvor i bufferen, hvilken shader-attribut det er*.

Shader-program To små C-lignende programmer skrevet i GLSL, der kører direkte på GPU'en: en *vertex shader* (1 gang pr. vertex/partikel) og en *fragment shader* (1 gang pr. pixel).

Vertex Et enkelt punkt i 3D-rum med tilhørende data. I dette projekt repræsenterer hver vertex én mulig elektronposition med attributterne position (x, y, z) og farve (r, g, b, a). Alt består af verticies⁶, en trekant består af tre vertecies, en quad (firkant) består af to trekanter, og derfor 6 vertecies and so on.

På Figur 3 ses en illustration af VBO og VAO.

⁶Vertex i flertal

6.3.2 Opsætning

```

1 self.vao      = glGenVertexArrays(1)
2 self.vbo_pos  = glGenBuffers(1)
3 self.vbo_col  = glGenBuffers(1)
4
5 glBindVertexArray(self.vao)
6
7 glBindBuffer(GL_ARRAY_BUFFER, self.vbo_pos)
8 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, None) # location=0, 3 floats
9 glEnableVertexAttribArray(0)
10
11 glBindBuffer(GL_ARRAY_BUFFER, self.vbo_col)
12 glVertexAttribPointer(1, 4, GL_FLOAT, GL_FALSE, 0, None) # location=1, 4 floats
13 glEnableVertexAttribArray(1)

```

Listing 3: VAO/VBO-opsætning i Engine.__init__.

Location 0 og 1 matcher `layout(location=0)` og `layout(location=1)` i vertex shaderen. På den måde ved GPU'en præcis, hvilken buffer der hører til hvilken input variabel.

6.3.3 Upload af partikler m. dynamisk buffer

Hver gang brugeren ændrer n , l eller m (eller slipper slideren efter at have ændret antal partikler), skal *hele* bufferen på GPU'en overskrives med nye data. Dette gøres med `upload_particles(self, positions, colors)` funktionen:

```

1 def upload_particles(self, positions, colors):
2     self.n_particles = len(positions)
3     pos_flat = positions.astype(np.float32).ravel()
4     col_flat = colors.astype(np.float32).ravel()
5     glBindBuffer(GL_ARRAY_BUFFER, self.vbo_pos)
6     glBufferData(GL_ARRAY_BUFFER, pos_flat.nbytes, pos_flat, GL_DYNAMIC_DRAW)
7     glBindBuffer(GL_ARRAY_BUFFER, self.vbo_col)
8     glBufferData(GL_ARRAY_BUFFER, col_flat.nbytes, col_flat, GL_DYNAMIC_DRAW)

```

Bemærk at `GL_DYNAMIC_DRAW` fortæller GPU'en om, at denne buffer vil blive *overskrevet mange gange*. Det er en lille optimering jeg opdagede under research i OpenGL dokumentationen. [4]

6.3.4 Shaders (GLSL)

```

1 #version 330 core
2 layout(location = 0) in vec3 aPos;
3 layout(location = 1) in vec4 aColor;
4 uniform mat4 uMVP;
5 out vec4 vColor;
6 void main() {
7     gl_Position = uMVP * vec4(aPos, 1.0);
8     gl_PointSize = 2.5;
9     vColor       = aColor;
10 }

```

Listing 4: Vertex shader. Transformerer hver partikel fra verdenskoordinater til skaermkoordinater.

```

1 #version 330 core
2 in vec4 vColor;
3 out vec4 FragColor;
4 void main() {
5     vec2 coord = gl_PointCoord - vec2(0.5);
6     float dist = length(coord);
7     if (dist > 0.5) discard;           // rundt udsnit
8     float alpha = (1.0 - smoothstep(0.2, 0.5, dist)) * vColor.a;
9     FragColor = vec4(vColor.rgb, alpha);
10 }

```

Listing 5: Fragment shader. Laver et firkantet point om til en blød, rund partikel med glød-effekt.

På Listing 4 og 5 ses de hhv. vertex shaderen og fragment shaderen. Vertex shaderen er ansvarlig for, hvor på vertexen skal dette punkt placeres. Den modtager en 3D position fra VBO'en og ganger den med MVP matrixen for at beregne den tilsvarende 2D skærmposition. Fragment shaderen er ansvarlig for, hvilken farve skal denne pixel have. Den modtager farven fra vertex shaderen og beregner den endelige pixelfarve, herunder den bløde cirkulære kant der gør hver partikel til en rund glødende prik i stedet for en firkant. Bemærk at disse to shaders, skrevet i GLSL er plukket fra forskellige stackoverflow tråde.

Det blev en rund partikel. Hvis man bare tegner `GL_POINTS` som de kommer, bliver de små *firkanter*. Derfor stødt jeg på funktionen `discard` i fragment shaderen. For hver pixel regnes afstanden til punktets centrum, og pixels udenfor radius 0,5 kasseres. `smoothstep` funktionen giver desuden en blød kant, så partiklen ser mere ud som en glød end en disk. Det gjorde en kæmpe forskel visuelt.

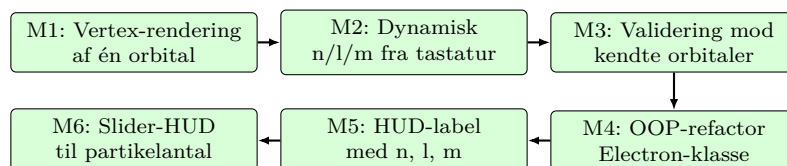
Additiv blending for glød. I `Engine.__init__` sættes:

```
glBlendFunc(GL_SRC_ALPHA, GL_ONE)    # additiv blending
```

Dette betyder, at overlappende partikler *lægges sammen* i stedet for at overskrive hinanden. Tæthedsregioner bliver derfor automatisk lysere, uden ekstra kode. Under HUD rendering skifter jeg til normal blending (`GL_ONE_MINUS_SRC_ALPHA`) og tilbage igen bagefter.

7 Udviklingsprocessen

7.1 Milepæle



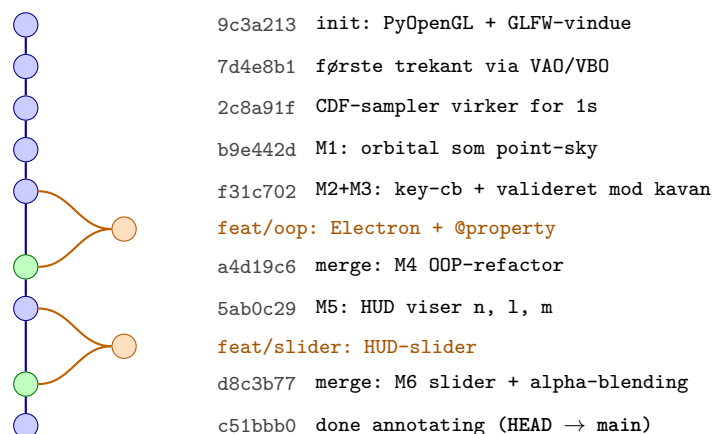
Figur 4: De seks faktiske milepæle i projektforløbet. Alle er afsluttet.

På Figur 4 ses nogle af de fremtredende milepæle under den itterative arbejdsproces. **M1** var det sværeste tekniske spring, altså at få *nogen som helst* punkt på skærmen via shaders krævede, at hele OpenGL konteksten, VAO, VBO, shader kompilering og MVP matrixen virkede samtidig.

Først efter **M1** var færdig, blev de øvrige milepæle forholdsvis lige til. De første 5 milepæle er desuden også **need-to-have** krav til projektet. Det kan derfor allerede ses at disse krav er opfyldt, og har været store skridt i processen.

7.2 Git

Under processen blev Git udnyttet til versionstyring. Der har været adskillige mange Git commits og pushes under vejs. Derfor er Git commit history alt for ulæselig og lang. Jeg har dog prøvet at optegne en lille alternativ med de store commits.



Figur 5: De store commits i projektets Git-historik, opstillet som en forenklet graf. Blå cirkler = **main**, orange = feature-branches (**feat/oop** og **feat/slider**), grøn = merge-commits. De mange små fix- og format-commits er udeladt for læsbarhed.

Programmets kode findes i følgende Git Repo <https://github.com/KhizarIrshadChaudhry/quantum-mechanical-atom-simulation>.

7.3 Udfordringer undervejs

De tre største udfordringer, som de så ud *for mig* (en der aldrig havde kodet direkte mod GPU før):

1. **Første møde med OpenGL.** Der er ingen `draw(x,y)` som i p5.js. I stedet skal man selv skrive shader kode, allokere buffers og binde attributter. Jeg brugte flere dage på bare at få ét eneste punkt på skærmen.
2. **VAO/VBO/shader pipelinen fra bunden.** Fejlmeddelelserne er meget lav niveau ("shader compile error" med en GLSL-line-nummer), og programmet kan tegne et *sort vindue* uden at give fejl, hvis man blot har glemt at binde VAO'en.
3. **Lav-niveau følelsen ved at allokere bytes.** Funktionen `glBufferData(GL_ARRAY_BUFFER, pos_flat.nbytes, pos_flat, GL_DYNAMIC_DRAW)` er et enkelt kald, men kræver forståelse for, hvorfor `.ravel()` og `.astype(float32)` er nødvendige. Ellers bliver bytes på GPU'en *misfortolket*, og man får underlige mønstre.

8 Test

For at konkludere om programmet har levet op til sine krav, som set i kravspecifikationen benyttes der nogle såkaldte *tests*. Testen har været *manuel og visuel*, da output er grafisk. For hver (n, l, m) -kombination har jeg sammenlignet med kendte orbitaldiagrammer fra Wikipedia. Tabellen nedenfor viser nogle af disse tests samt den hhv. observation og resultat.

Krav	Test	Observation	Resultat
K1	Start programmet i 1s ($n=1, l=0, m=0$).	En kugleformet sky omkring origo.	OK
K2	Sæt $n=2, l=5$ via tastatur (ulovlig).	1 klampes til 1.	OK
K3	Tryk W flere gange.	Sky vokser og får nye noder.	OK
K4	Træk slider fra min til max.	Antal partikler opdateres ved slip.	OK
K5	Tryk W/E/R.	HUD viser korrekt n, l, m .	OK
K6	2p-orbital ($n=2, l=1$).	Lobes har lys kerne, mørk ydre.	OK
K7	1M partikler i 3d.	Sjældne områder halvgennemsigtige.	OK
K8	Træk muse + scroll.	Orbit-kamera + zoom virker.	OK

Nogle edge cases jeg har testet:

- Slider ved minimum (10.000 partikler) $\rightarrow$ stadig genkendelige former.
- Slider ved maksimum (1.000.000 partikler) $\rightarrow$ framerate ~ 50 FPS, hvilket er acceptabelt.
- $n = 6, l = 5 \rightarrow$ komplekse flerlobede strukturer, dog stadig synlige som forventet.
- Tryk S når $n = 1 \rightarrow$ forbliver $n = 1$ (fysiske regler overskrides ikke: $n \geq 1$).

9 Diskussion

Den **modulære opdeling** af projektet har i praksis vist sin værdi, f.eks. da jeg kunne refaktorere **Electron**-klassen (M4) uden at røre ved **engine.py**, og slideren (M6) kunne tilføjes uden at røre ved **sampler.py**. Det er én af de gevinster ved OOP og separation of concerns, som er svær at mærke i små programmer, men som blev tydelig her. Tilsvarende leverer den **CDF baserede sampling** matematisk korrekte fordelinger, og fordi den er vektoriseret via **numpy**, er den rigtig hurtig til 1M partikler. Samples beregnes på CPU'en én gang pr. tastetryk, og sendes derefter til GPU'en.

Der er dog også tydelige begrænsninger. Selvom **numpy** er hurtigt, er **sampler.py** stadig et **CPU bundet flaskehals** ved 1M partikler ($\sim 0,8$ s genereringstid). Alternativt ville GPU compute shader kunne gøre dette i realtid, men ville kræve et helt nyt lag af OpenGL kode. Udover det, er **farvemappingen en approksimation**. Tætheden udregnes ved at tælle partikler i et 60^3 voxel-grid (**np.bincount**)⁷, og ved meget lave partikelantal bliver farverne støjede, løsningen er ret simpel (flere partikler), men antallet af voxels kunne adaptivt justeres afhængigt af antal partikler. Endelig understøtter programmet **kun hydrogen**. Det er hardcodet til hydrogenlignende atomer med én elektron, og flere elektroner ville kræve Pauli princippet og Slater determinanter, hvilket er uden for projektets "scope".

Fremadrettet ser jeg tre decideret konkrete forbedringsforslag. En **pre sampling cache** kunne

⁷Inspiret af en Reddit opslag.

forudberegne de ofte valgte (n, l, m) kombinationer, så skiftet mellem orbitaler blev instant. **Asynkron resampling** i en worker-tråd ville sikre, at UI'en aldrig fryser, selv når brugeren trækker slideren til maksimum. Og **flere visualiseringstilstande**, fx en "slice through" med et 2D-farvemap, ville være bedre egnet til undervisning.

10 Konklusion

Alle otte krav i min kravspecifikation (K1–K8) er opfyldt. Problemformuleringens fire delspørgsmål er besvaret: (1) CDF sampling oversætter $|\psi|^2$ til diskrete **numpy**-punkter, (2) OOP via en **Electron**-klasse med **@property** og **set_kvantumtal** håndhæver kvantetalsreglerne ét sted, (3) en VAO/VBO/shader-pipeline er bygget fra bunden i PyOpenGL og renderer op til 1.000.000 partikler i realtid, og (4) resampling udskydes til mouse-release og holder sig på CPU'en, så render-loopet aldrig blokerer mere end nødvendigt.

Det største faglige udbytte har været forståelsen af OpenGL pipeline, samt low-level GPU funktioner og egenskaber. Jeg har bevæget mig fra det "tegne-som-på-papir-niveau (p5.js) til faktisk at skrive GLSL shaders og binde GPU-buffers. Det føles som et niveau-hop i forståelsen af, hvad der *faktisk* sker, når en computer tegner på en skærm. Det er den største ting jeg tager med videre. Projektet har givet mig muligheden for at dyrke de tre ting jeg er mest passioneret over. Nemlig fysik, matematik og programmering.

11 Perspektivering

Hvis projektet tages seriøst, har den naturlige videreudviklinger inden for flere retninger. **Didaktisk** set kunne programmet bruges i HTX-fysikundervisningen som et interaktivt læringsværktøj på Fysik A (supplerende stof), eller evt. på 1. semester uni. Der findes ikke mange af lignende simulationer der er nem at bruge. **Teknisk**: flytte samplingen til en GPU compute shader ville give realtids interaktion, skifte fra point sprites til volumen raymarching ville give smukkere elektronskyer. Der er selvfølgelig mange måder at repræsentere det samme på. Hvert har sine fordele og ulemper. **Fagligt**: en udvidelse til fler-elektron-atomer åbner for orbital hybridisering og molekylorbitaler. Evt. en direkte bro til Kemi A stof, der igen kunne benyttes til undervisning på HTX.

Litteratur

- [1] Chaudhry, K. I. (2025). *Kvantemekanisk Sammenfiltrering*. SOP, NEXT Sukkertoppen HTX.
- [2] kavan010. *Hydrogen Simulator*. YouTube og tilhørende GitHub-repository.
- [3] The Khronos Group. *OpenGL 3.3 Core Specification & Reference Pages*. <https://www.khronos.org/opengl/wiki/>.
- [4] PyOpenGL dokumentation. <https://pyopengl.sourceforge.net/>.
- [5] Wikipedia. *Inverse transform sampling*. https://en.wikipedia.org/wiki/Inverse_transform_sampling.
- [6] Wikipedia. *Laguerre polynomials* og *Spherical harmonics*.
- [7] Wikipedia. *Cantor function*. https://en.wikipedia.org/wiki/Cantor_function#Properties.
- [8] Chaudhry, K. I. (2024). *Simulering af Gravitation og visuelisering af rum-tid Gravity*. Programmering B & Fysik A simuleringsprojekt i p5.js.
- [9] StackOverflow-tråde om GLSL point-sprite rendering, `GL_PROGRAM_POINT_SIZE` og `gl_PointCoord`.

Supplerende brug af generel internetsøgning, spørgsmål på forum, og AI hjælp til fejlfinding i shader-kompilering og ortografisk projektion matrix (row-major vs. column-major).

A Bilag - Kodebasen

De vigtigste filer (main.py, electron.py, sampler.py, particles.py, uddrag af engine.py og camera.py) vedlægges eksamen som separat zip fil. Programmets kode findes også i følgende GitHub repo <https://github.com/KhizarIrshadChaudhry/quantum-mechanical-atom-simulation>. Her ses kun de mest centrale uddrag for bedre overblik:

A.1 main.py - event-loopet

```
1 while not glfw.window_should_close(engine.window):
2     glfw.poll_events()
3     engine.draw(camera)
```

A.2 sampler.sample_orbital() - centralt samplings kald

```
1 def sample_orbital(n, l, m, n_particles=50_000):
2     r_max = 10 * n**2
3     r_vals = np.linspace(1e-6, r_max, 3000)
4     r_prob = _radial_prob(n, l, r_vals)
5     r_samp = _cdf_sample(r_vals, r_prob, n_particles)
6
7     theta_vals = np.linspace(1e-6, np.pi - 1e-6, 2000)
8     theta_prob = _angular_prob(l, m, theta_vals)
9     theta_samp = _cdf_sample(theta_vals, theta_prob, n_particles)
10
11     phi_samp = np.random.uniform(0.0, 2.0 * np.pi, n_particles)
12
13     r_pm = r_samp * A0_PM
14     x = r_pm * np.sin(theta_samp) * np.cos(phi_samp)
15     y = r_pm * np.cos(theta_samp)
16     z = r_pm * np.sin(theta_samp) * np.sin(phi_samp)
17     return np.stack([x, y, z], axis=1).astype(np.float32)
```

A.3 particles.compute_colors() - densitet til farve

```
1 def compute_colors(positions, n_bins=60):
2     coords = positions.copy()
3     mins, maxs = coords.min(axis=0), coords.max(axis=0)
4     ranges = maxs - mins
5     ranges[ranges == 0] = 1.0
6
7     voxel = ((coords - mins) / ranges * (n_bins - 1)).astype(int)
8     voxel = np.clip(voxel, 0, n_bins - 1)
9
10    flat = (voxel[:, 0] * n_bins**2 +
11           voxel[:, 1] * n_bins +
12           voxel[:, 2])
13    counts = np.bincount(flat, minlength=n_bins**3).astype(np.float32)
14    density = counts[flat] / counts[flat].max()
15
16    rgba = _colormap(density).astype(np.float32) # inferno colormap
17    rgba[:, 3] = np.clip(density * 2.0, 0.05, 0.95) # alpha
18    return rgba
```